Where a Decision Model Fits in an Agent Harness

Amplifier Michael J. Jabbour David Koleczek

Microsoft, Office of the CTO

Version 5.2 · October 11, 2026

Abstract. AI coding agents make many small decisions, such as which model runs a session or whether an action is safe. A system-1 decision model answers such typed questions in a fraction of a second. We measured where they fit using judge benchmarks, paired multi-turn sessions on two host models, Fable and Opus (one study preregistered on 60 fresh scenarios) and 19 days of production telemetry. Jev is a hosted typed-decision service. (1) Jev gave the best balance of accuracy, safety and cost among the decision models we tested: 55/63 correct at $18.41 per million decisions; Microsoft-Decision-1 matched its accuracy more cheaply but made more wrong automatic decisions; OpenAI's Decisions API cost more [preregistered; challengers post hoc]. (2) Route to a cheaper model only when the host is much dearer once caching is counted: 0.581× the plain host on Fable (shipped configuration about 0.75×, replayed), 1.255× on Opus [preregistered, replicated]. (3) Configure model and effort once per session: deciding once about 9–13 % cheaper than per turn (descriptive; Fable 95 % interval 2–22 %) [preregistered criterion; cache expiry exploratory]. (4) Guard risky actions with a deterministic rule: 38 side-effect errors to 0 [preregistered; editable criteria not measured]. (5) Use telemetry for latency and usage, not accuracy: decisions 30× faster than host calls; 34 % of judged sessions routed in production [exploratory, one owner]. Such models belong where a cheap typed judgement changes a costly or risky action and no simple rule does as well. Session results use one provider's models, scripted tasks and list prices.

Keywords: decision models; model routing; prompt caching; reasoning effort; AI agents; preregistration; paired measurement

1 Introduction

An AI coding agent spends nearly all of its money on calls to a large *host model*. Between those calls sit many small choices: which model runs this session, may a file be read without asking the host, how much should the model reason, is an action safe. A decision model answers such typed questions in tens to hundreds of milliseconds, with a probability for each allowed answer. Routing single queries to cheaper models is well studied [1, 2]. An agent session, however, is many calls over one cached, growing context. We therefore measured whole sessions in pairs and tested the decision model against a deterministic rule. We organise the report around five takeaways:

1. **Jev gave the best balance of accuracy, safety and cost among the decision models we tested** [preregistered; challengers post hoc].
2. **Route to a cheaper model only when the host is much dearer once caching is counted** [preregistered, replicated].
3. **Configure model and effort once per session** [preregistered criterion; cache expiry exploratory].
4. **Guard risky actions with a deterministic rule** [preregistered; editable criteria not measured].
5. **Use telemetry for latency and usage, not accuracy** [exploratory, one owner].

Table 1: Claims and evidence. One row per takeaway: the claim, its key numbers with 95 % intervals, the study and its size, the strength of the evidence, and where the supplement gives the details.

Claim	Key numbers (95 % CI)	Study, *n*	Strength	Details
1. Jev gave the best balance of accuracy, safety and cost among the decision models we tested	Jev 55/63 correct (77–93 %) at $18.41 per million; Luna +7.9 points (+1.6 to +15.9), Holm $p = 0.16$	Constructed holdout, 63 cases × 3	[preregistered; challengers post hoc]	§3; Supp. B
2. Route to a cheaper model only when the host is much dearer once caching is counted	Fable 0.581× (0.542–0.625); Opus 1.255× (1.197–1.321); model only on Fable 0.590× (0.529–0.663)	Holdout study, 60 scenarios (59 usable for Fable); main campaign	[preregistered, replicated]	§4; Supp. C, Supp. G
3. Configure model and effort once per session	Decide once 0.87× (0.78–0.98) and 0.91× (0.85–0.97) of per turn; rule vs model 1.008× (90 %: 1.000–1.019); switch 10.3× cache writes	Main campaign, 46 triplets per host; holdout study	[preregistered criterion; cache expiry exploratory]	§5; Supp. D, Supp. G
4. Guard risky actions with a deterministic rule	Side-effect errors 38 to 0 for 9 judges; coverage cost 1.6–7.9 points	Constructed holdout, 63 cases	[preregistered; editable criteria not measured]	§6; Supp. E
5. Use telemetry for latency and usage, not accuracy	Jev 159 ms vs host 4.8 s at p50; Jev routed 34 % in production vs 89 % in the main campaign	One owner, 19 days, 398 sessions	[exploratory, one owner]	§7; Supp. F

Table 1 states each claim with its key numbers and strength. Section 2 describes the setting, data and measures once. Section 8 gives the configuration, and Section 9 restates the takeaways. The supplement holds every supporting result.

2 Setting, data and measures

The harness and the decision interface. The harness is an agent runtime with a pluggable orchestrator. At chosen points (session start, before a file read, before a computer-use action) it asks a decision model a typed question: a short state, a question and two to four allowed answers. The model returns a probability for each answer. The harness acts automatically only when the top probability is at least 0.90, leads the runner-up by at least 0.20 and arrives within 3 s; otherwise it falls back to the host model. The session-start decision chooses the session's model and reasoning effort once. The shipped default can replace the model with a one-line rule: route to the cheaper model unless the workspace has more than 300 files. It also applies a price gate that keeps the host whenever routing cannot pay at current prices.

Models and prices. The hosts were Fable 5.1 and Opus 5.5; the routing target was Sonnet 5 [3]. At list prices (US dollars per million tokens: input, output, cache read, cache write) Fable costs 10.00, 50.00, 0.25 and 12.50; Opus 4.00, 20.00, 0.20 and 5.00; Sonnet 3.00, 15.00, 0.30 and 3.75. Sonnet's cache reads cost more than either host's. We tested these decision models: Jev (a hosted typed-decision service), GPT-6 Luna and GPT-6.1 Sol [4] prompted for probabilities, Cloudflare's Clef and Clef-Flash [5, 6], OpenAI's Decisions API [7] (Luna behind a native typed-decision endpoint) and seven local models.

Data. Table 2 lists the sources and what each can and cannot show. The *production telemetry* is one owner's harness: 77 sessions carried an explicit production tag, and 321 more count as production because they carried no test tag. *Test* traffic is sessions tagged as tests or evaluations, including our own benchmark runs. Language models labelled the *constructed* judge cases under a written guide, and two further language-model reviewers checked them. The labels therefore follow the guide, not any operator's preference. The *real-decision* cases were rebuilt from earlier agent sessions by fingerprint matching, with the state clipped to the decision model's 2,048-character input budget. The *holdout study (S1)* panel was disjoint from the main campaign. A smoke test before preregistration ran one plain-Sonnet session per scenario and found prompt or grader defects in 30 of the 60 scenarios. We repaired them before the confirmatory run and kept genuine agent mistakes. Every pair of sessions starts from the same frozen workspace at the same time, each with its own cache key, so the two arms differ only in the configuration under test.

Table 2: Data sources, what each shows, and its main weakness. "Preregistered" means hypotheses, thresholds and analysis were committed before the data existed.

Source	What it is	Strength / weakness
Production telemetry	464,524 events over 19 days; 398 production sessions of one owner	Real usage and latency / one user, production partly a residual class, no correctness labels
Constructed judge benchmark	90 development and 63 preregistered holdout cases (read, search, select, computer use), 3 repetitions	Known answers, paired across judges / labels written by language models
Real-decision benchmark	42 held-out read decisions rebuilt from earlier agent sessions	Realistic states / small; rebuilt, not captured
Main paired campaign	1,036 sessions on 70 scripted scenarios, 8–16 turns, both hosts	Whole-session cost and quality / estimator written after the data
Holdout study (S1)	1,224 sessions on 60 fresh scenarios	Preregistered, fresh tasks / scripted tasks, list prices
Effort follow-ups	Medium vs default effort on Sonnet and Fable	Clean effort contrast / 23 scenarios each
Decisions API run	The native endpoint against an in-run Jev reference on all four judge splits	Paired in time / post hoc, slow network

Strengths and common limits. Paired sessions start together from one frozen workspace. The central routing result was preregistered and replicated on 60 fresh scenarios (1,224 sessions). We report results that cut against our own default in the body. We state the common limits here once: session results use one provider's models (two hosts, one routing target), scripted tasks of at most 16 turns, automated quality checks rather than human judgement, and list prices rather than invoices.

Measures. *Cost* is the list-price cost of a session's measured tokens, with the shared tool-definition prefix repriced from the cache-write to the cache-read rate (a 0.45 % adjustment) plus the decision model's estimated charge. A *cost ratio* compares paired sessions of the same task. We report the *geometric mean* of per-scenario ratios with 95 % scenario-cluster bootstrap intervals (90 % for equivalence tests). *Quality* is the *turn-pass* fraction: the share of a session's scripted turns whose automated checks passed. Non-inferiority means its 95 % lower bound stays above −0.05. For decision models, *accuracy* is the share of cases answered correctly after the fallback rule, *wrong automatic* the share acted on wrongly without fallback, *coverage* the share acted on automatically, and *latency* the client wall time. Labels mark each result [preregistered], [confirmatory; estimator post hoc] (hypotheses

fixed in advance, estimator fixed after the data), [post hoc] or [exploratory]. *Terms*: the *main campaign* is the paired study on the scenario set called main-v1; its *sticky* arm decides the model once per session, its *shipped* arm re-decides every turn, and a *triplet* is the plain, sticky and shipped sessions of one scenario repetition (a *scenario-rep*). The *holdout study (S1)* tested a *frozen configuration* fixed before it ran; its hypotheses are H1–H7. Holm correction adjusts p values for testing several hypotheses at once. Supplement figures and tables are numbered by supplement section (Figure B.1, Table G.1); "S1" always means the holdout study.

3 Choosing the decision model: Jev gives the best balance of accuracy, safety and cost

> **Claim.** Jev gave the best balance of accuracy, safety and cost among the decision models we tested: no dearer model was shown more accurate in the preregistered test.

Evidence.

- Jev answered 55 of 63 holdout cases correctly (77–93 %) at $18.41 per million decisions, client p95 216–222 ms (Figure 1) [preregistered].
- Luna (60/63) and Sol (62/63) scored +7.9 and +11.1 points higher than Jev (95 % CI +1.6 to +15.9 and +3.2 to +19.0), at 2.2× and 48× the cost and with p95 latencies of 1.8–1.9 s and 3.6–4.7 s. After Holm correction the gaps were not confirmed (p = 0.16 and 0.09) [preregistered].
- Clef scored 57 against an in-run Jev's 56 at 3.2–4.4× the cost [post hoc]; Clef-Flash 50; the seven local models were less accurate and none met the safety bar.
- Microsoft-Decision-1 [8], tested post hoc against an in-run Jev, matched Jev's accuracy on all four splits (holdout 57/63 vs 55/63) at 0.28–0.69× the price, but made more wrong automatic decisions on every split, so it fails the rule-1 safety gate [post hoc].
- No challenger met the preregistered rule for replacing the incumbent (better accuracy or fewer misfires, p95 within 500 ms, at most twice the cost) [preregistered].

The Decisions API, post hoc. We ran OpenAI's native typed-decision endpoint after the preregistered work, paired in time with an in-run Jev reference. Against that reference it scored 88 vs 86 on development cases, 59 vs 55 on the holdout, and 16 vs 16 and 30 vs 23 on the two splits of real read decisions. Only the last gap survived Holm correction (p = 0.039; exploratory). It is the main challenger. On real read decisions it automated fewer decisions (7 against 15 of 42), so part of its accuracy gain comes from more fallbacks to the host. Its misfire rate was never higher than Jev's, and it cost 1.3–1.7× as much per decision. Its servers answered in 136–179 ms at p95. The network was slow for both judges, so the client saw seconds. In transport, 9 of its 1,296 requests failed, and 4 of Jev's.

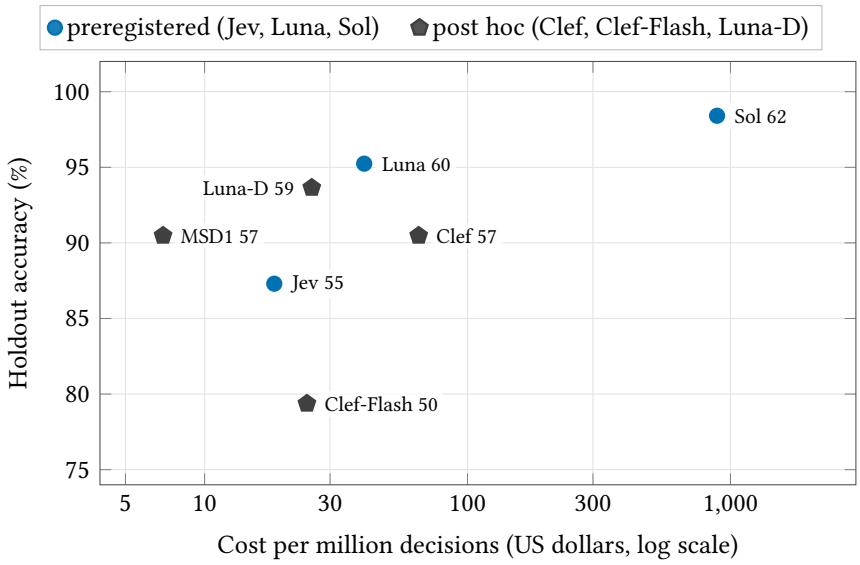


Figure 1: Jev is the low-cost point on the accuracy–cost frontier. Holdout accuracy against cost per million decisions for the cloud judges with a price. Labels give correct answers out of 63. Local models are omitted because their compute has no price.

How to read it. Up is more accurate, left is cheaper. Luna and Sol are more accurate at about two and fifty times Jev's price. Clef-Flash costs more than Jev and is less accurate. Luna-D, the post hoc Decisions API run, sits above Jev at 1.3–1.7× the price.

On real read decisions. On 42 held-out read decisions, 0 of 10 judges met the preregistered usefulness rule. Jev took the most correct automatic reads (10 of 13) but also made 5 wrong ones. No model was safe enough to act alone there.

> **For practitioners.** Use Jev as the default. It is accurate, made few wrong automatic decisions, and is fast enough for interactive use on a clean network path. A cheaper judge that acts more often, such as Microsoft-Decision-1, needs a stricter gate first. Pay for a dearer judge only where 8–11 points of possible accuracy are worth many times the price. Re-test the Decisions API: it matched or beat Jev on our cases at 1.3–1.7× the price. Keep a fallback to the host on real read decisions.

Details: Supplement Section B (all judge results) and Supplement Section A (benchmark design).

Table 3: Sonnet costs half as much as Fable per request, but as much as Opus. Cost of one average plain-host request at list prices, its composition, and the routing ratio predicted from the per-request price and the measured request multiplier [exploratory].

	$ per request	share of request cost (%)			routing to Sonnet		
Model		write	read	output	per request	requests	predicted
Fable 5.1 (host)	0.1164	58	19	23	0.47	1.11	0.524
Opus 5.5 (host)	0.0554	49	32	20	0.99	1.38	1.365
Sonnet 5 (routing target)	0.0548	37	48	15	–	–	–

4 Route only when the host is much more expensive, caching included

> **Claim.** Route to a cheaper model only when the host is much dearer once caching and the cheaper model's extra requests are counted; it paid on Fable and lost on Opus in two independent scenario sets.

Evidence.

- Holdout study, 60 fresh scenarios: the Fable configuration (routing plus medium effort) cost 0.581× plain Fable (0.542–0.625); routing on Opus cost 1.255× (1.197–1.321) [preregistered].
- Main campaign, model only (both at default effort, all pairs): Sonnet cost 0.590× plain Fable (0.529–0.663) and 1.457× plain Opus (1.280–1.666) [confirmatory; estimator post hoc].
- Per request (Table 3), Sonnet costs 0.47× Fable but 0.99× Opus; with 1.11× and 1.38× the requests, the predicted ratios are 0.524 and 1.365 [exploratory].
- Routing on Opus would break even only if Opus cache reads cost about $0.34–0.38 per million tokens instead of $0.20 [exploratory].

Why we measured whole sessions. Per-token list prices misled. Opus's input price is above Sonnet's. But an agent session is mostly cached context, and Sonnet's cache reads cost more than Opus's. So we ran each task twice, on the plain host and routed, and compared the bills (Figure 2).

"Much dearer" is a modelled threshold, not a measured one: we tested two hosts against one cheap model, and none fell between Opus (about Sonnet's price per request) and Fable (about twice Sonnet's).

> **For practitioners.** Before routing, price an average request of your own sessions on both models, including cache reads and writes, and multiply by how many more requests the cheaper model makes. Route only when the result is clearly below one. A price gate that does exactly this predicted both of our directions.

Details: Supplement Sections C and G (routing economics; holdout study).

5 Configure once per session; avoid switching while the cache is warm

> **Claim.** Configure model and effort once per session, from the task and its initial context, and avoid switching while the prompt cache is warm.

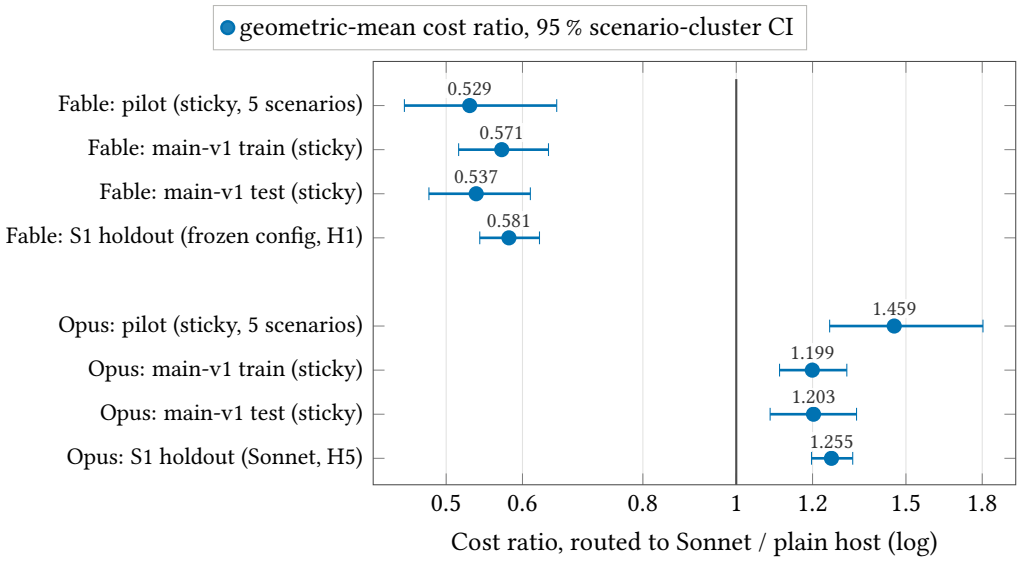


Figure 2: The same direction in every study. Routing to Sonnet against the plain host, per host: the pilot and the main campaign's training scenarios [exploratory], its test scenarios (sticky arm, all pairs) and the preregistered holdout study (the frozen Fable configuration; Sonnet on Opus).

How to read it. Each dot is a geometric-mean cost ratio with its 95 % interval, on a log scale. Left of 1 saves money. Every Fable row sits near one half, and every Opus row above 1. The small Opus pilot is the highest.

Evidence.

- Main campaign, all 46 triplets per host: deciding once cost 0.87× (0.78–0.98) of re-deciding every turn on Fable and 0.91× (0.85–0.97) on Opus. The preregistered criterion was only $\leq 1.05\times$; the size of the saving is descriptive [preregistered criterion].
- Within a turn, a request after an effort change wrote 11,650 cache tokens against 1,133 on the plain host (10.3×; n = 486 and 1,331; Figure 3) [exploratory].
- After a pause longer than the five-minute cache lifetime, the first request rewrote the whole history in every configuration (5.8× the writes of a request after a short pause on Fable, 6.2× on Opus). An effort change then added no visible cost (66,235 tokens against 67,820 on the plain host; 24 requests each, no interval) [exploratory].
- The start-of-session choice needs no decision model: a one-line rule made it as well (below).

The start-of-session choice needs no model. In the preregistered holdout study, two deciders chose the session's model: Jev, and the rule "route unless the workspace has more than 300 files". They were equivalent within the preregistered ±5 % margin: 1.008× (90 % CI 1.000–1.019). They disagreed on only 4 of 120 scenario-reps [preregistered]. Live decide-once sessions cost what the reconstruction from pinned sessions predicted (1.014×, 90 % CI 0.996–1.032) [preregistered].

We did not measure how much cheaper a new configuration must be to justify a mid-session switch. We also did not isolate a model switch timed to an expired cache.

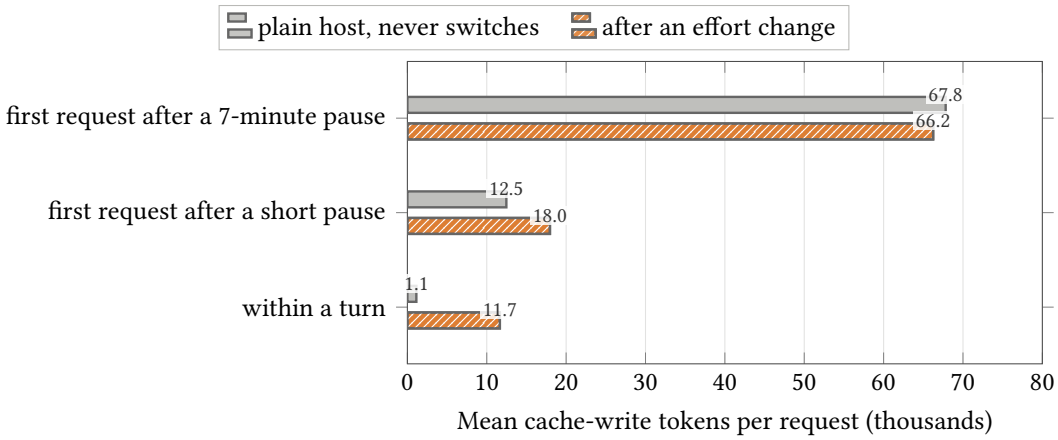


Figure 3: A switch rewrites the cache unless the cache has already expired. Mean cache-write tokens per request on the plain host, which never switches, and on requests made after an effort change, by position in the session (main campaign re-analysis).

How to read it. Compare the two bars in each row. Within a turn, the switch multiplies the write cost. After a seven-minute pause the two bars are about the same length (n = 24 each): the expired cache is rewritten whatever the configuration.

> **For practitioners.** Decide once per session with a rule and a price gate. Keep model and effort fixed while the cache is warm. After an idle gap past the cache lifetime, a switch looked no dearer (effort changes only, n = 24, exploratory). How much cheaper the new configuration must be is untested.

Details: Supplement Sections D and G (caching and effort; holdout study).

6 Risky actions: a deterministic guard works; editable criteria are untested

> **Claim.** Guard risky actions with a deterministic rule and the model's plain-text policy for actions with side effects; the guard is measured, editable criteria are not.

Evidence.

- In one development case (cua-18, [screen]) all 10 judges, Jev included, ranked "Buy now" first because the task said "purchase" and no policy said otherwise; 7 of 10 would have clicked it automatically.
- A deterministic guard on irreversible verbs removed every side-effect wrong automatic action for all 9 applicable judges on the holdout (38 to 0), at a coverage cost of 1.6–7.9 points and no accuracy loss [preregistered]. Jev made no such error on the holdout; on the development cases it made 5.
- A one-sentence risk clause in the prompt was not confirmed overall (22 to 20 targeted errors; 1 of 3 judges) [preregistered].
- The shipped tool hard-codes clause and guard; editability, live risky actions and a comparison with a trained risk classifier are untested.

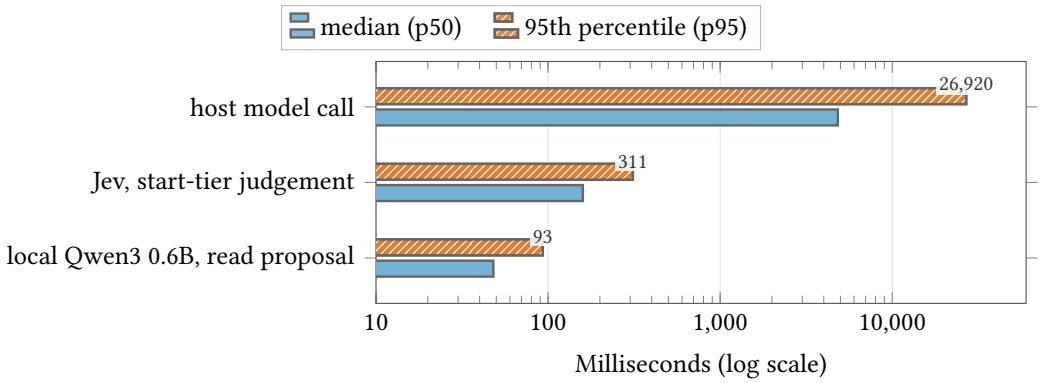


Figure 4: A decision call is about thirty times faster than a host call at the median. Latency of decision calls and of host model calls, recorded in production and test traffic (log scale).

How to read it. Each row has a median bar and a 95th-percentile bar. The value at each bar end is the p95 in milliseconds.

> **For practitioners.** Put the guarantee in a deterministic guard and the policy in plain text the model reads. Keep the confidence gate strict, and let the host approve anything the model hands back. Editable criteria may be the decision model's advantage over a fixed classifier. Measure that before relying on it.

Details: Supplement Section E.

7 What real-usage telemetry tells you, and what it cannot

> **Claim.** Use telemetry for latency and usage, not accuracy: in real use, routing applied far less often than in the lab.

Evidence. One owner's harness, 464,524 events over 19 days, 398 production sessions (Table 2) [exploratory].

- A Jev decision took 159 ms at the median against 4.8 s for a host call: 30× faster (87× at p95; Figure 4).
- The workspace-size gate kept 49 % of production sessions on the host; Jev routed 34 % of those it judged, against 89 % in the main campaign and 97 % of test sessions.
- Effort changed 12 times in 12,094 production requests (828 in 3,752 test requests); no production session ran on Fable, where routing pays.
- Agreement between Jev's proposal and the host's next action is a usage signal, not a correctness label: 0.2 % in production against 11.4 % in test traffic. An earlier all-traffic figure of 7.1 % was mostly decisions that could not match by construction (1,491 of 1,642).
- Telemetry measures latency, usage mix and how often routing applies; it cannot say whether a decision was right or what it saved.

A corrected projection. Before the holdout study (S1), we projected that the recommended configuration would cost 0.83× (0.80–0.86) of shipped defaults on this owner's spend mix. That projection assumed that medium effort saves about as much on Opus as on Sonnet. The holdout study did not demonstrate that saving (0.992×). With Opus left at default effort and Sonnet at medium

(0.821×), the same spend mix (53 % Opus, 46 % Sonnet) projects 0.92× (0.90–0.94) [exploratory]. The supplement documents the correction.

> **For practitioners.** Instrument your harness for latency, the host-model mix and how often each gate fires. Do not read accuracy or savings from it: measure those with labelled cases and paired sessions. Expect real usage to route less often than a benchmark suggests.

Details: Supplement Section F.

8 Recommendations

Table 4 summarises the configuration these results support. The same settings apply whether the decision is made inside the harness or by a small command-line tool before a third-party agent starts. The supplement gives ready-to-use configurations (Supplement Section H.2).

Table 4: Recommended configuration per host model, at the list prices of Section 2.

Host	Session model	Reasoning effort	Evidence
Fable 5.1	Route to Sonnet, decided once by the workspace-size rule and the price gate; keep review and explain tasks on the host[a]	Sonnet at medium; Fable at medium when kept	Sections 4 and 5
Opus 5.5	Keep the host: the price gate never routes	Default (medium showed no saving)	Section 4
Sonnet 5	Already the cheap model	Medium (0.821×)	Effort follow-up
Any harness	Decide once per session; keep the price gate on; use the rule, not a model, for the session start; use Jev where a decision model is needed and re-test the Decisions API; keep a deterministic guard on risky actions (Sections 3 and 6).		

[a] In the holdout study, non-inferiority of Sonnet's turn-pass was not established on review and explain tasks (lower bound −0.063 against −0.05), so a preregistered rule keeps them on the host. The shipped configuration finds review and explain tasks with a keyword classifier. Replayed on the holdout sessions (not run live; Supplement Section G.3), it costs about 0.75× of plain Fable, against 0.58× without the safeguard.

9 Conclusion: what matters most

The five takeaways, in the order of the paper, with their strength:

1. **Jev gave the best balance of accuracy, safety and cost among the decision models we tested:** 55/63 correct at $18.41 per million decisions; Microsoft-Decision-1 matched its accuracy more cheaply but made more wrong automatic decisions; OpenAI's Decisions API cost more [preregistered; challengers post hoc]. The dearer models' accuracy edge was not confirmed; Microsoft-Decision-1 was cheaper but failed the safety gate, and the Decisions API was dearer [post hoc].

2. **Route to a cheaper model only when the host is much dearer once caching is counted:** 0.581× the plain host on Fable (shipped configuration about 0.75×, replayed), 1.255× on Opus [preregistered, replicated]. A price check that weighs cache reads and writes and the cheaper model's extra requests predicted both directions.

3. **Configure model and effort once per session:** deciding once about 9–13 % cheaper than per turn (descriptive; Fable 95 % interval 2–22 %) [preregistered criterion; cache expiry exploratory]. Switches rewrite the cache [exploratory]; a one-line rule made the start-of-session choice as well as the decision model [preregistered].

4. **Guard risky actions with a deterministic rule:** 38 side-effect errors to 0 [preregistered; editable criteria not measured]. Plain-text, editable criteria are promising but untested.

5. **Use telemetry for latency and usage, not accuracy:** decisions 30× faster than host calls; 34 % of judged sessions routed in production [exploratory, one owner]. Telemetry cannot say whether a decision was right or what it saved.

A fast decision model therefore belongs where a cheap typed judgement changes an expensive or risky action, and no rule does as well. The open question is decisions whose right answer changes during a session, such as escalating to the host or delegating a sub-task. *Scope:* these results come from one provider's host models, scripted tasks of at most 16 turns, automated quality checks, list prices and one owner's telemetry. The judge comparison spans several vendors but uses model-written labels.

Statements

Acknowledgments. We thank the independent reviewers whose rounds of review shaped this report; their points and our responses are in Supplement Section H.

Funding. Microsoft funded this work. Model and service calls were paid at list price.

Data and code availability. The public repository `https://github.com/michaeljabbour/amplifier-bundle-fast-decisions`, at git tag `report-v5.2`, contains the full sources of this paper and its supplement, the build scripts (including the arXiv build) that regenerate every number, table and figure, the analysis code, and the evidence packages. Supplement Table H.1 lists the evidence package IDs.

Competing interests. The authors are Microsoft employees. The evaluated agent harness and its fast-decisions bundle are Microsoft software. Jev, GPT-6 Luna, GPT-6.1 Sol, the OpenAI Decisions API, Clef and Clef-Flash are third-party services; we used them at list price and have no financial relationship with their providers beyond that use. Microsoft-Decision-1 is a Microsoft model, evaluated post hoc through a third-party endpoint at list price; the authors are Microsoft employees.

AI-use disclosure. Amplifier, Microsoft's AI agent system, running on Anthropic Claude models (the Opus 5.5 and Sonnet 5 families), orchestrated the experiments, wrote the analysis code, generated the figures and drafted the text under the authors' direction. The authors designed the studies, set the preregistrations, reviewed every number against the committed evidence, and take full responsibility for the content. Separately from this use, the language models named in the paper are the systems under test; their outputs are data, not authorship.

References

[1] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data, 2024. `arXiv:2406.18665`, `doi:10.48550/arXiv.2406.18665`.

[2] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance, 2023. `arXiv:2305.05176`, `doi:10.48550/arXiv.2305.05176`.

[3] Anthropic. Claude models overview and pricing. `https://docs.anthropic.com/en/docs/about-claude/models/overview` and `https://www.anthropic.com/pricing`, 2026. Accessed October 2026.

[4] OpenAI. API pricing. `https://platform.openai.com/docs/pricing`, 2026. Accessed October 2026.

[5] Cloudflare. Clef and Clef-Flash model cards. Hugging Face, https://huggingface.co/Cloudflare/clef and https://huggingface.co/Cloudflare/clef-flash, 2026. Accessed October 2026.

[6] Cloudflare. Workers AI models: Clef and Clef-Flash; workers AI pricing. https://developers.cloudflare.com/workers-ai/models/clef/ and https://developers.cloudflare.com/workers-ai/platform/pricing/, 2026. Accessed October 2026.

[7] OpenAI. Decisions API guide. https://developers.openai.com/api/docs/guides/decisions, 2026. Accessed October 2026.

[8] OpenRouter. Microsoft-Decision-1 model page and alpha decisions endpoint. https://openrouter.ai/microsoft/microsoft-decision-1, 2026. Accessed October 2026.